GUIDE

How to Make Block Themes Better with Theme JSON

Last Updated May 28, 2025 • 12 minute read

By Iain Poulson

Senior Product Manager

View Profile

Key points:

- WordPress theme settings were previously scattered across files, but are now mostly held within the theme.json file.
- Its structure covers a wide range of settings, including the schema, version, custom templates, and patterns.
- You can modify theme.json directly with code for advanced control or visually through the Site Editor for visual customization.
- Theme.json integrates with block development frameworks like ACF Blocks, allowing custom blocks to inherit your theme's global style system.

For years, WordPress theme settings were scattered – defined in *functions.php*, buried in PHP hooks, or hardcoded into stylesheets. It worked, but barely. Now, there's *theme.json* – a single, declarative file that brings order to the chaos. It centralizes theme configuration and design tokens, letting developers define

global settings like colors, typography, layout, and spacing in one place.

The real payoff is in the block editor: *theme.json* powers the Global Styles interface, so designers can tweak visual styles directly – without risking layout breakage. Developers, meanwhile, keep control over what can and can't be changed. The output is lean: only the CSS you specify, with no bloat and fewer fights over selector priority.

More than just a convenience, *theme.json* is now the standard foundation of all core themes. It's tightly integrated with the WordPress editor and aligned with where the platform is headed.

Let's explore how it works and why it's fundamental for modern WordPress development.

What is theme.json in WordPress?

In WordPress, *theme.json* is a single configuration file placed at the root of a theme. It defines global settings and styles for both the block editor and the front end using a structured JSON format.

This file brings consistency and predictability to theme development by centralizing what used to be scattered across *functions.php*, `add_theme_support()` calls, and custom stylesheets.

Using JSON makes configurations easier for WordPress to parse and for developers and teams to share. It's the same reason we use it to configure ACF Blocks.

It works in both classic and block themes; it's technically optional, but almost every modern theme includes it as a standard practice.

This file powers the Global Styles interface in the editor, giving users a visual way to customize colors, typography, layout, and spacing without needing to write code.

Behind the scenes, it serves as the single source of truth for theme design, feeding four layers of style data – core defaults, block defaults, theme values, and user customizations – which WordPress merges at runtime.

One of its biggest benefits is reducing the need for excessive custom CSS. Only the necessary styles are generated, in the correct cascade order, helping to avoid conflicts and unnecessary overrides.

Beyond styles, *theme.json* is also extensible: you can define templates, template parts, patterns, and block style variations under structured keys.

Understanding the structure of theme.json

Here's a breakdown of the main components you'll define in a *theme.json* file – and what each one controls.

Schema

The `$schema` key is an optional but recommended top-level entry in theme.json. It links to the official WordPress JSON Schema – typically something like `"$schema": "https://schemas.wp.org/trunk/theme.json"` – or a version-specific path such as */wp/6.5/theme.json*.

Adding this enables advanced features in modern code editors and IDEs, including autocompletion, real-time validation, and inline documentation for available keys and values.

It has zero effect on how WordPress renders the theme; it exists purely to improve the developer experience. Some schema validators require it to be listed before the version key, so placing it at the top is best practice.

While WordPress functions fine without it, skipping it means you'll lose helpful editor features that can prevent syntax errors and speed up configuration.

Version

The `version` key is a required top-level field in theme.json that tells WordPress which version of the spec your theme uses:

- `"version": 1` enables the original feature set introduced in WordPress 5.8, allowing basic global styles and settings.

- `"version": 2`, the default since WordPress 5.9, adds support for block-specific settings, layout presets, and deeper editor integration.
- `"version": 3`, available in WordPress 6.6+ and recent Gutenberg releases, introduces enhanced style inheritance and lets you define custom CSS variables directly within *theme.json*.

Each version expands what the theme can configure. If WordPress doesn't recognize the version number, it quietly falls back to version 1 behavior. That means no errors, but also no access to newer features. To prevent this, only update the version key when your theme's minimum WordPress requirement matches the version of the spec you're targeting.

Settings

The `settings` object is the core of theme.json, defining what users can customize in the block editor and setting the design system for your theme. This is where you enable or disable editor features and register design tokens like colors, fonts, spacing, and more.

Settings follow a hierarchy:

- **Global** settings live at the root of the settings object.
- **Elements** (settings > elements > link, button, heading, etc.) let you target HTML elements directly.
- **Blocks** (settings > blocks > "core/paragraph") allow block-specific overrides.

Common setting groups include color, typography, spacing, layout, border, dimensions, appearanceTools, and custom.

These settings populate the Global Styles UI, giving users predefined options while preventing option overload. You control what can be changed and what stays locked, reducing inconsistencies across a site.

Here's an example of what you might find in this segment:

```
"settings": {
  "color": {
    "palette": [
    { "slug": "brand", "color": "#0057ff", "name": "Brand Blue" }
    ]
```

```
    },
    "typography": {
      "fontFamilies": [
      { "slug": "inter", "fontFamily": "Inter, sans-serif", "name": "Inte
      ]
    }
  }
```

This code registers a custom color and font, which then appear in the editor's design tools and are used throughout the site.

Custom templates

The customTemplates key lets you register additional page templates that appear in the Template dropdown within the editor. These are useful for offering alternate layouts – like a no-header landing page – without requiring users to touch code.

Each template entry is an object with three fields:

- `"name"` matches the filename (e.g., *templates/landing-no-header.html* in a block theme or *page-landing-no-header.php* in a classic theme).
- `"title"` is the human-readable name shown in the editor.
- `"postTypes"` defines where the template is available, usually `["page"]`.

Take the code sample below:

```
"customTemplates": [
  {
    "name": "landing-no-header",
    "title": "Landing – No Header",
    "postTypes": ["page"]
  }
]
```

Once registered, this template appears in the Template selector for the defined post types, allowing authors to apply a custom layout without modifying code. It works in both block and classic themes, as long as the file structure matches the name.

Template parts

The `templateParts` key registers reusable layout sections – like headers, footers, or sidebars – for use in the Site Editor. These parts live in the parts/ directory of your theme and are referenced by name.

Each entry includes:

- `"name"` is the slug matching a file in parts/ (e.g., *parts/header.html*).
- `"title"` is the label shown in the editor.
- `"area"` defines the intended usage zone (`"header"` , `"footer"` , `"sidebar"` , etc.).

Here's an example:

```
"templateParts": [
  {
    "name": "header",
    "title": "Site Header",
    "area": "header"
  }
]
```

Registering parts this way exposes them in the Site Editor's "Replace" panel, making it easier to swap in alternate versions. It also enables translation of part titles and clarifies their purpose for both users and developers.

These parts can then be referenced in full-page templates using block markup like `<!-- wp:template-part {"slug":"header"} /-->` .

Patterns

The `patterns` key allows you to bundle pre-designed block patterns with your theme by referencing their slugs. These can come from the official WordPress Pattern Directory or be registered by plugins, e.g.:

```
"patterns": [
  "two-column-cta",
  "pricing-table-staggered"
```

```
]
```

This setup ensures those patterns are readily available in the editor, giving users curated starting points for building pages while keeping your block theme templates neat and flexible.

When the theme is activated, WordPress fetches the specified patterns so they appear in the Block Inserter under the Patterns tab.

This is a lightweight way to provide layout ideas or reusable design sections without hardcoding them into templates. You can also combine bundled patterns with synced (formerly reusable) patterns created in the Site Editor, letting users update all instances of a pattern site-wide from one place.

How to work with theme.json

There are two ways to modify *theme.json*: by editing the file directly or by using the Full Site Editor (FSE) to adjust styles visually.

For simple changes like updating colors or fonts, the FSE is often safer and faster, especially for non-developers – it reduces the risk of breaking anything with a syntax error.

For site owners or editors, the FSE is perfect for rebranding, quick visual fixes, or updates to a live site without needing server access.

Developers, on the other hand, may use the FSE to prototype styles, then export those settings to commit into code.

Editing the file directly also allows for advanced configuration, like adding new palette values or using keys not exposed in the UI, and ensures the design system stays version-controlled and portable.

Supercharge Your Website With Premium Features Using ACF PRO

Speed up your workflow and unlock features to better develop websites using ACF Blocks and Options Pages, with the Flexible Content, Repeater, Clone, Gallery Fields & More.

Explore Features

View Pricing

PRO Features

ACF Blocks

Options Pages

PRO Fields

Repeater

Flexible Content

Gallery

Clone

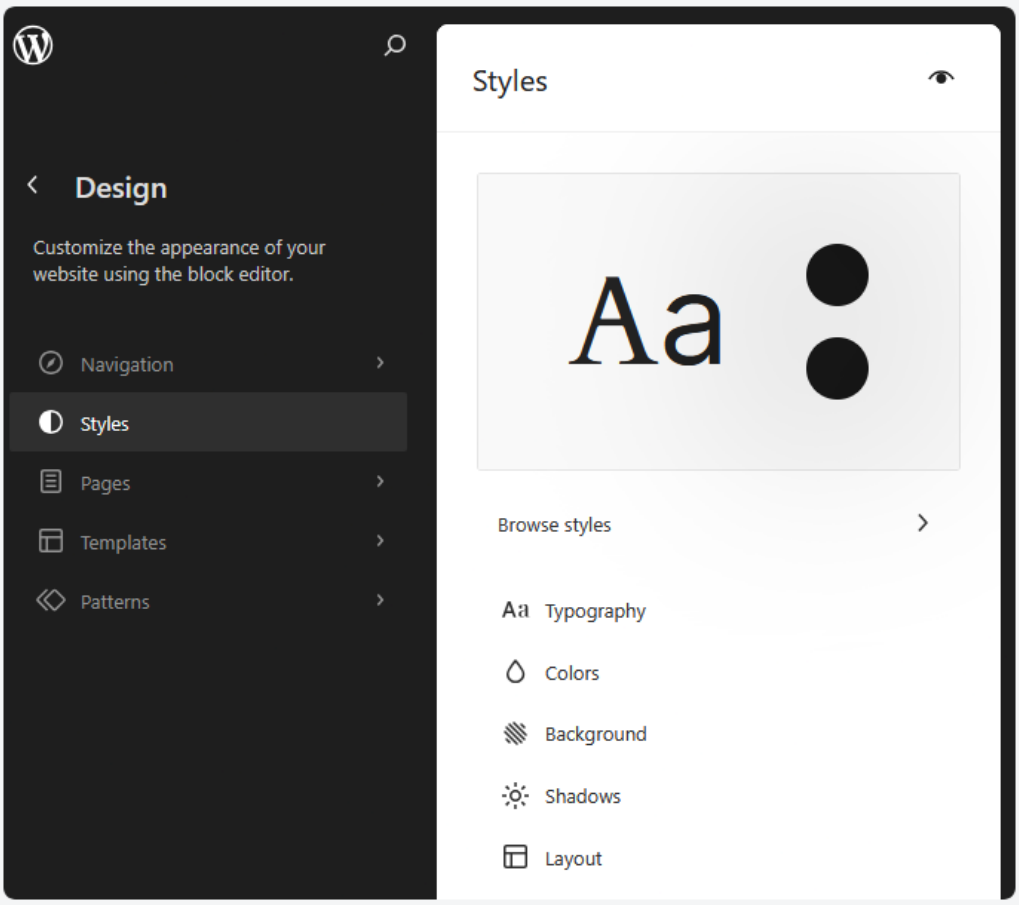
Method 1. Making customizations via the Site Editor

The WordPress Site Editor will update your *theme.json* with changes you make via the UI, meaning you don't have to touch code for minor to intermediate edits. For this walkthrough, we're going to focus on a couple of styles, specifically the typography and colors.

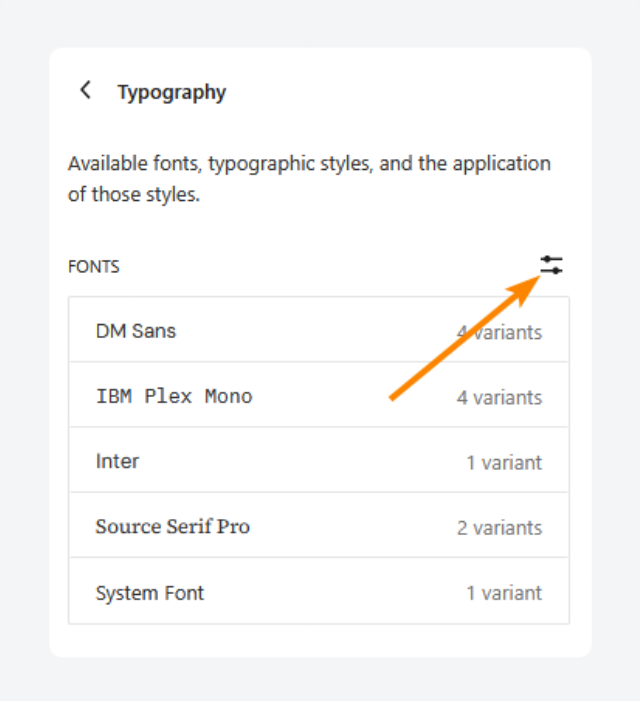
Let's start with the all-important typography:

1. From your dashboard, go to *Appearance* > *Editor*.

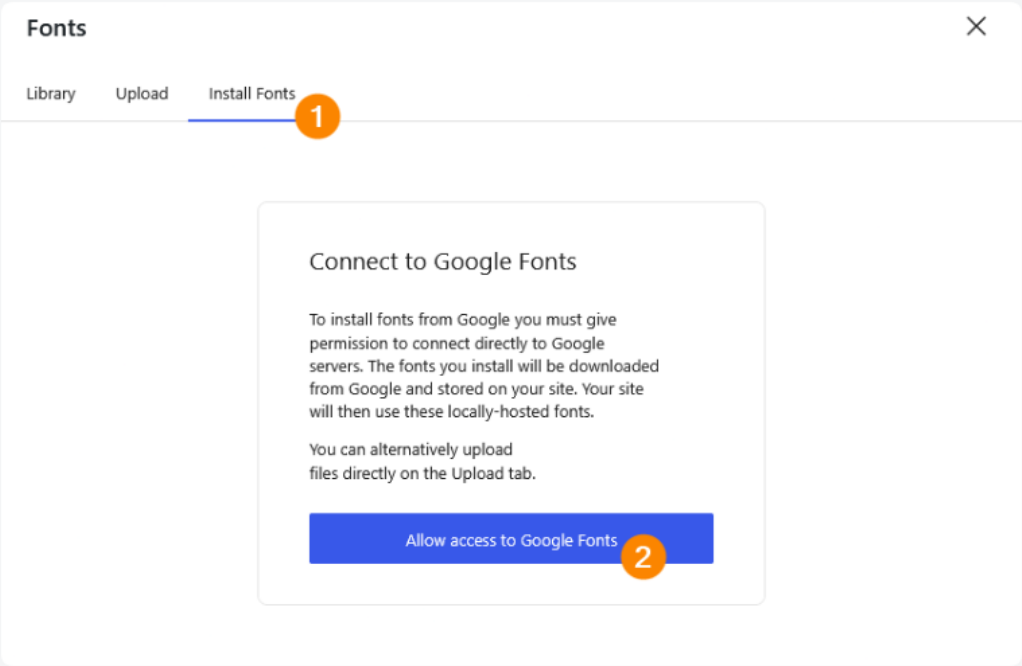
2. Click on *Styles*, where you'll see *Typography* and *Colors* available to customize.



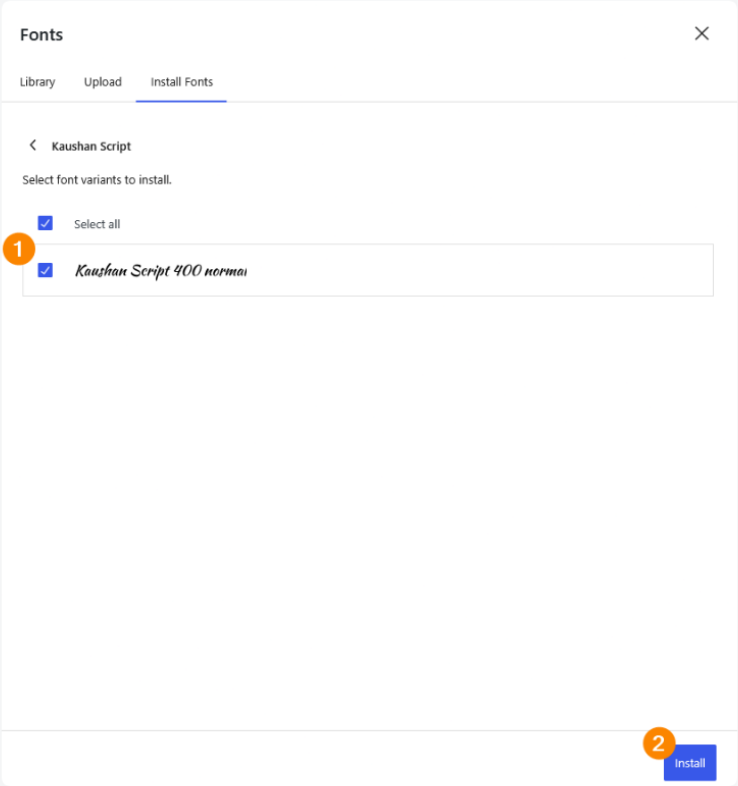
3. Click on *Typography*, then click the sliders icon next to *Fonts*.



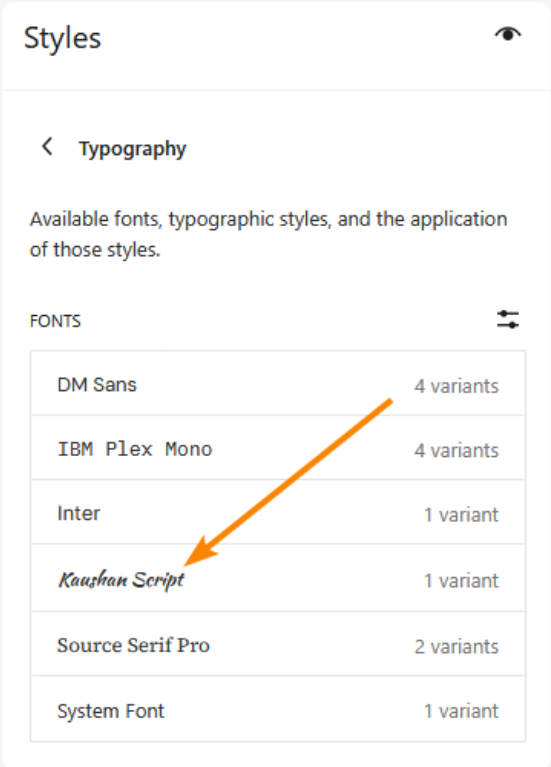
4. Switch to the *Install Fonts* tab, then click *Allow access to Google Fonts*.



5. Find your preferred font from the list. We're going to use Kaushan Script, which is really *out there*, but that makes its presence obvious.
6. Select the variants you want to use and click *Install*.

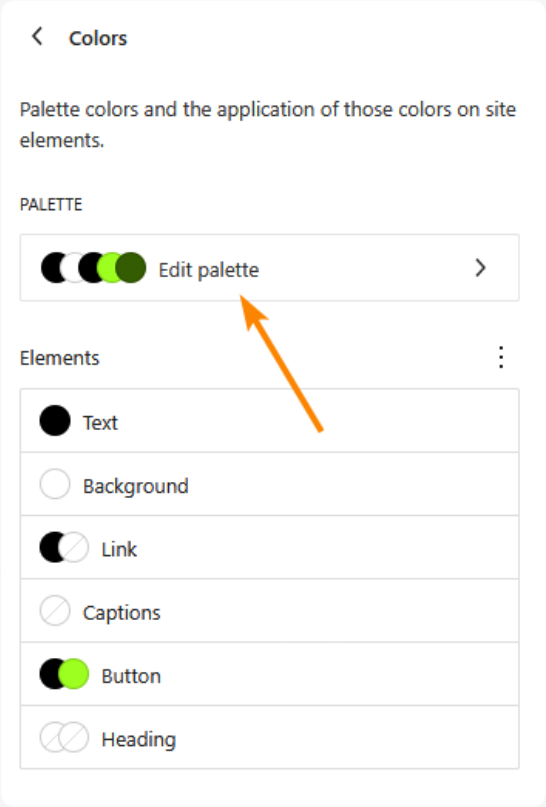


7. Close the popup and check that the font appears in the *Typography* editor.

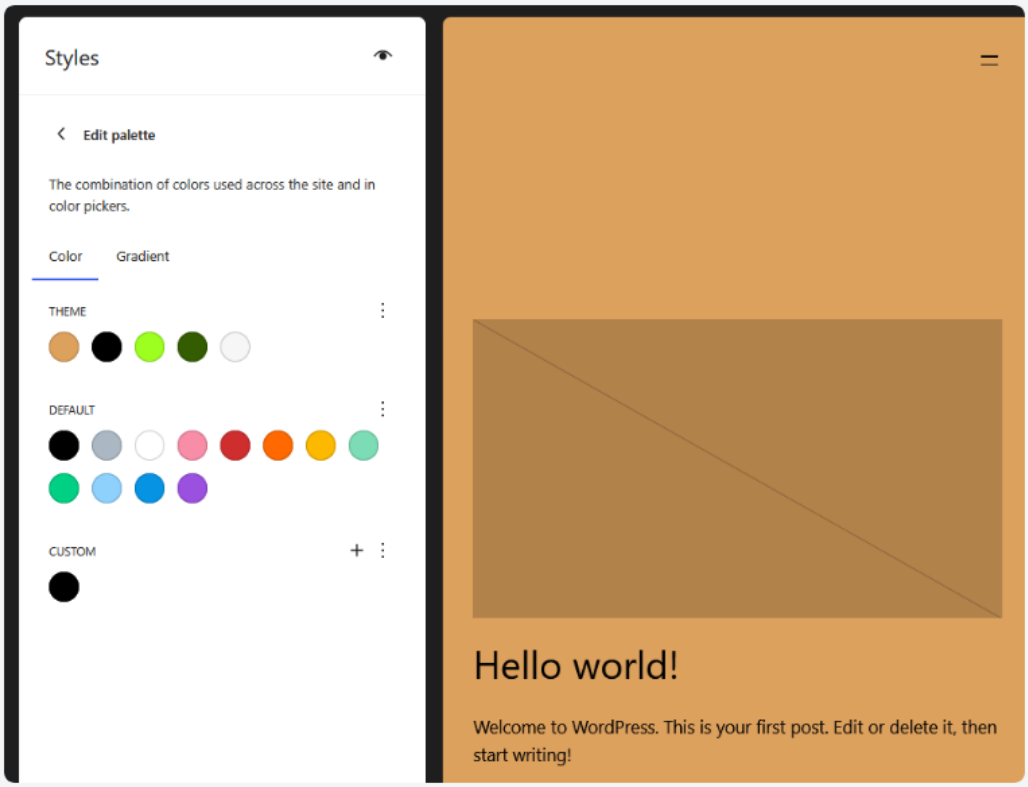


Now, we're going to edit the colors:

1. Go to *Styles* > *Colors*.
2. Click *Edit palette*.

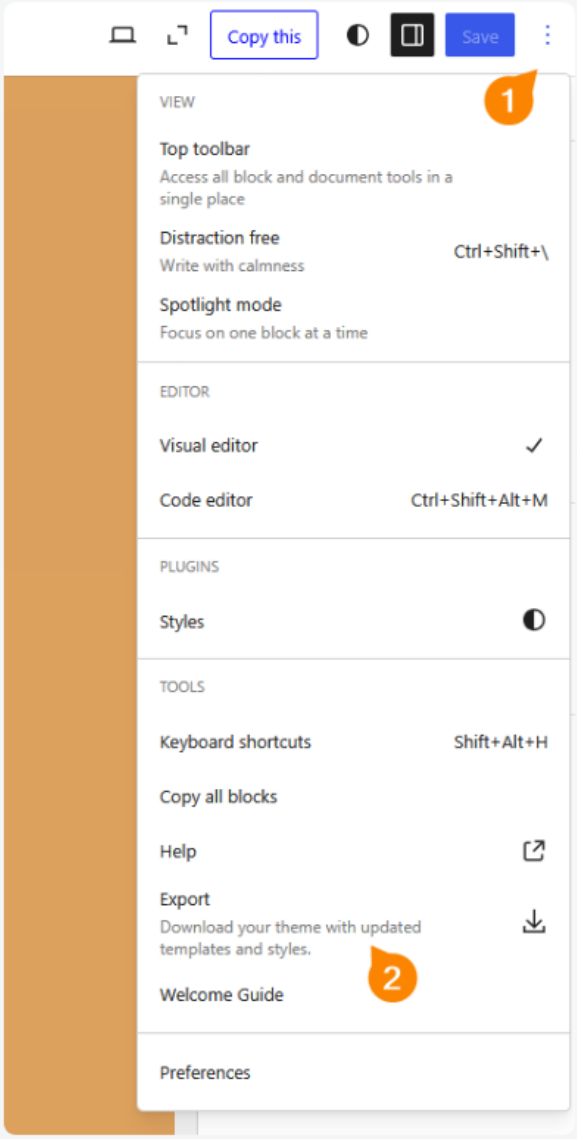


3. Click the first color under *Theme* and enter the hex value for what you want to replace it with. We're going to use `#dda15e`, which is a sort of orange brown.



Finally, we're going to save the changes and take a look at the *theme.json*:

1. Click anywhere on the preview of the site to open the Gutenberg Editor.
2. Click *Save* at the top right to save the changes you made to the colors and typography. This action will update your *theme.json* file.
3. Click the three dots next to the *Save* button, then click *Export*.



This will download your theme, with all its files, as a zip archive. In the root folder of this archive is *theme.json*. Open it in a code or text editor and look for your custom font under `“settings”:` > `“typography”:` > `“fontFamilies”:` :

```
},
{
    "fontFace": [
        {
            "fontFamily": "\"Kaushan Script\"",
            "fontStyle": "normal",
            "fontWeight": "400",
            "src": "https://[redacted].com/wp-content/uploads/fonts/vm8vd
        }
    ],
    "fontFamily": "\"Kaushan Script\", cursive",
    "name": "Kaushan Script",
    "slug": "kaushan-script"
}
```

Then look for your custom color under `“settings”:` > `“color”:` > `“palette”:` :

```
],
"settings": {
    "appearanceTools": true,
    "color": {
        "palette": [
            {
                "color": "#dda15e",
                "name": "Base",
                "slug": "base"
            },
```

Method 2. Advanced customizations with code

There's a lot you can do with *theme.json* via the Site Editor, meaning you only need to edit it for more advanced settings. Or when you're building a theme from scratch, of course.

Batch editing is one example of an advanced use case, where editing the file directly is more practical. Some of the things you can do here include:

1. Creating a global link style, where one rule touches every hyperlink in every block:

```
"styles": {
  "elements": {
    "link": { "color": { "text": "var(--wp--preset--color--brand)" } }
  }
}
```

2. Setting a uniform block gap site-wide. Set it once, and it cascades everywhere instead of tweaking dozens of templates:

```
"styles": { "spacing": { "blockGap": "1.5rem" } }
```

Performance optimization is another area where the Site Editor just can't cut it. You can:

3. Restrict color and spacing tools so WordPress ships a lighter stylesheet:

```
"settings": {
  "color":  { "custom": false },
```

```
  "spacing":{ "units": ["rem","px"], "padding": false }
}
```

4. Predetermine content widths to avoid oversizing images and keep layout calculations fast:

```
"settings": { "layout": { "contentSize": "720px", "wideSize": "1200px"
```

5. Toggle fluid typography to generate a single `clamp()` rule per size instead of multiple media queries:

```
"settings": { "typography": { "fluid": true } }
```

Using ACF Blocks with theme.json

As we stated in an earlier section, we love JSON because it provides a much easier and readable way to structure data.

When creating a custom ACF Block, you'll configure it using a *block.json* file. To speed up development, you can inherit styles from *theme.json* by adding the following code to *block.json*:

```
"supports": {
  "color": true,
  "spacing": true,
  "typography": true
}
```

This will result in a sidebar that shows your theme's color palette, spacing scale, and fonts for you to pick when editing your custom block in Gutenberg. It's a faster, more scalable method for displaying custom field data on the frontend.

Take your WordPress theme to the next level with ACF

Mastering *theme.json* is a major step toward modern WordPress theme development, but it's just the beginning.

To truly unlock the full potential of your theme, pairing it with ACF takes things further. With ACF Blocks, you can build your own custom blocks that integrate with the block editor – and yes, they can inherit and respect your *theme.json* styles for consistent design.

ACF also lets you structure your site with custom fields and custom post types, giving you more control over how content is created, displayed, and managed. ACF helps you scale and manage modular layouts – be it for a portfolio, a business site, or a complex content platform.

If you're serious about creating flexible, client-friendly themes with a clean separation between design and data, explore ACF PRO today.

Supercharge Your Website With Premium Features Using ACF PRO

Speed up your workflow and unlock features to better develop websites using ACF Blocks and Options Pages, with the Flexible Content, Repeater, Clone, Gallery Fields & More.

Explore Features

View Pricing

PRO Features

ACF Blocks

Options Pages

PRO Fields

Repeater

Flexible Content

Gallery

Clone

By Iain Poulson
Senior Product Manager

View Profile

Iain is the Product Manager for Advanced Custom Fields. He has a long history of building and growing WordPress plugins. Moonlights as a PhpStorm evangelist.

TAGGED IN

ACF ACF Blocks WordPress

CONTINUE READING

Blog Archive / Step-by-Step: Enabling WooCommerce HPOS for Better Performance

Related Posts

WordPress Block Theme Development Step by Step

Learn to build modern WordPress block themes from scratch with our step-by-step guide. Master theme.json, templates, and ACF integration for fully customized...

Building WordPress Custom Themes From The Ground Up

Discover how to create unique WordPress themes from scratch. Our guide covers structure, responsive layouts, and leveraging WordPress functions for full customization.

Transform C into Streaml

Stop breakin deployments database sec workflows ac ACF Local JS

July 23, 2025

March 12, 2025

September 05,

For plugin support, please contact our support team directly, as comments aren't actively monitored.

Sign up for the latest Advanced Custom Fields news, updates, and developer tutorials

First name*

Last name*

Email*

Submit

© 2025 WPENGINE, INC. ALL RIGHTS RESERVED.

[1]WP Engine is a proud member and supporter of the community of WordPress® users. The WordPress® trademarks are the intellectual property of the WordPress Foundation, and the Woo® and WooCommerce® trademarks are the intellectual property of WooCommerce, Inc. Uses of the WordPress®, Woo®, and WooCommerce® names in this website are for identification purposes only and do not imply an endorsement by WordPress Foundation or WooCommerce, Inc. WP Engine is not endorsed or owned by, or affiliated with, the WordPress Foundation or WooCommerce, Inc.